



Wasm 生态介绍

蔡威 (Iceber Gu)





Content 目录

01 Wasm 发展历程

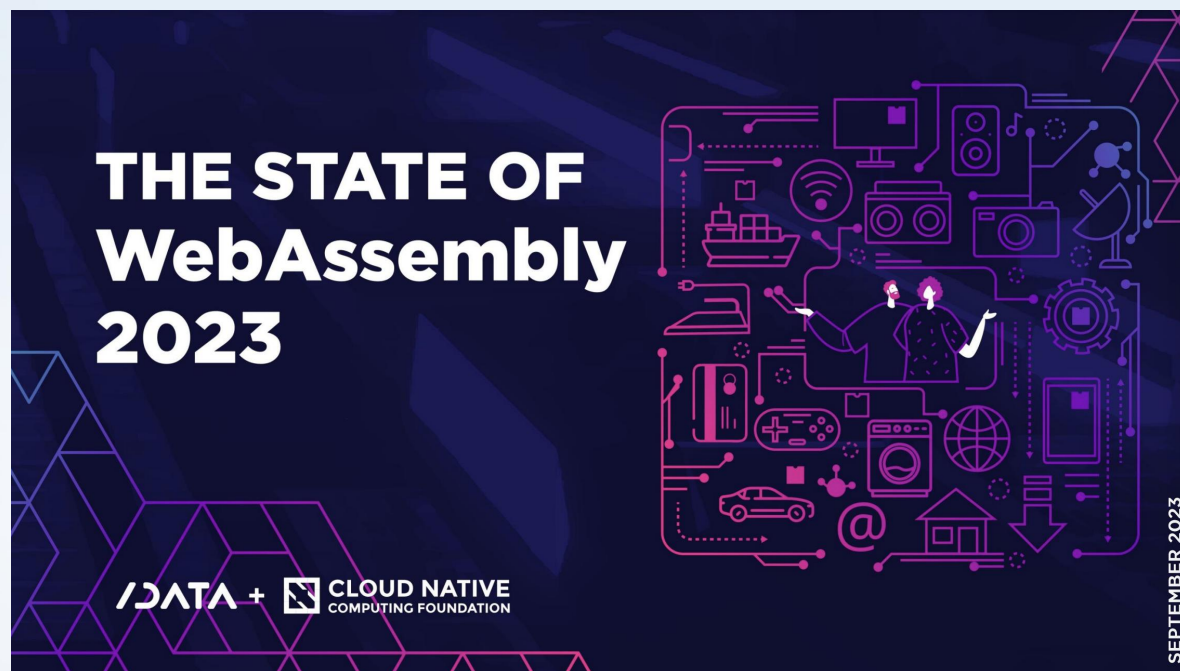
02 Wasm 介绍

03 Wasm 探索 Web 外的发展

04 Wasm 生态&组织

版权声明

分享内容中存在引用以下报告的信息



Tech · 18 October 2023 · 6 min read

The State of WebAssembly 2023

<https://www.cncf.io/wp-content/uploads/2023/09/The-State-of-WebAssembly-2023.pdf>

<https://blog.scottlogic.com/2023/10/18/the-state-of-webassembly-2023.html>



Part 01

Wasm 发展历程

根源：解决 JavaScript 在浏览器中的性能问题

2008：JavaScript JIT：

NaCL (Native Client) :

- 接近原生的性能
- 运行在沙箱中
- 架构绑定，无法跨平台

PNaCL (Portable Native Client) :

- 接近原生的性能
- 运行在沙箱中
- 跨平台
- 开发难度，成本等问题

Emscripten: 以 JavaScript 为编译目标，使 C/C++ 运行在浏览器中

asm.js: 作为 **Emscripten** 的编译目标，在 JavaScript 的基础上实现强类型，提供对于 JIT 的利用

但是从格式以及语法规则上依然还是 JS，

WebAssembly: 为浏览器而生的语言/格式

WebAssembly (WASM) 是由 W3C 制定的一种轻量，高效，安全，多语言，跨平台，并且可以运行在 Web 浏览器中的全新格式



WebAssembly: 为浏览器而生的语言/格式

WebAssembly (WASM) 是由 W3C 制定的一种轻量，高效，安全，多语言，跨平台，并且可以运行在 Web 浏览器中的全新格式





Part 02

Wasm 介绍

基于栈的虚拟机的二进制格式

Rust Code

```
#[no_mangle]
pub fn test(a: i32, b: i32) -> i32 {
    if b < 10 {
        a + b + 2
    } else {
        a + 3
    }
}
```

二进制格式 —— **.wasm**

000160	60	72	6E	61	69	0A	B1	95		01	7A	02	00	0B	15	00	20
000170	00	20	01	6A	41	02	6A	20		00	41	03	6A	20	01	41	0A
000180	48	1B	0B	31	01	01	7F	23		00	41	10	6B	22	02	24	00

Text 格式 —— **.wat**

```
(func $test (type 4) (param i32 i32) (result i32)
  local.get 0
  local.get 1
  i32.add
  i32.const 2
  i32.add
  local.get 0
  i32.const 3
  i32.add
  local.get 1
  i32.const 10
  i32.lt_s
  select)
```

\$ cargo build --target wasm-unknown-unknown --target release

\$ wasm2wat ./test.wasm -o test.wat

基于栈的虚拟机的二进制格式

Sections

```
test.wasm:      file format wasm 0x1

Sections:

  Type start=0x0000000a end=0x00000069 (size=0x0000005f) count: 15
  Function start=0x0000006b end=0x000000e6 (size=0x0000007b) count: 122
  Table start=0x000000e8 end=0x000000ed (size=0x00000005) count: 1
  Memory start=0x000000ef end=0x000000f2 (size=0x00000003) count: 1
  Global start=0x000000f4 end=0x0000010d (size=0x00000019) count: 3
  Export start=0x0000010f end=0x00000142 (size=0x00000033) count: 5
  Elem start=0x00000144 end=0x00000165 (size=0x00000021) count: 1
  Code start=0x00000169 end=0x000004c1a (size=0x000004ab1) count: 122
  Data start=0x000004c1d end=0x000005236 (size=0x00000619) count: 1
```

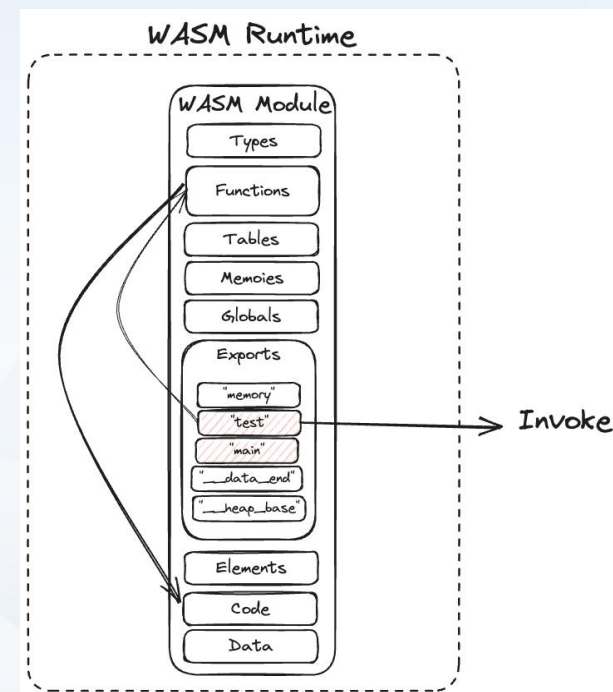
```
$ wasm-objdump -h ./test.wasm
```

Export Section

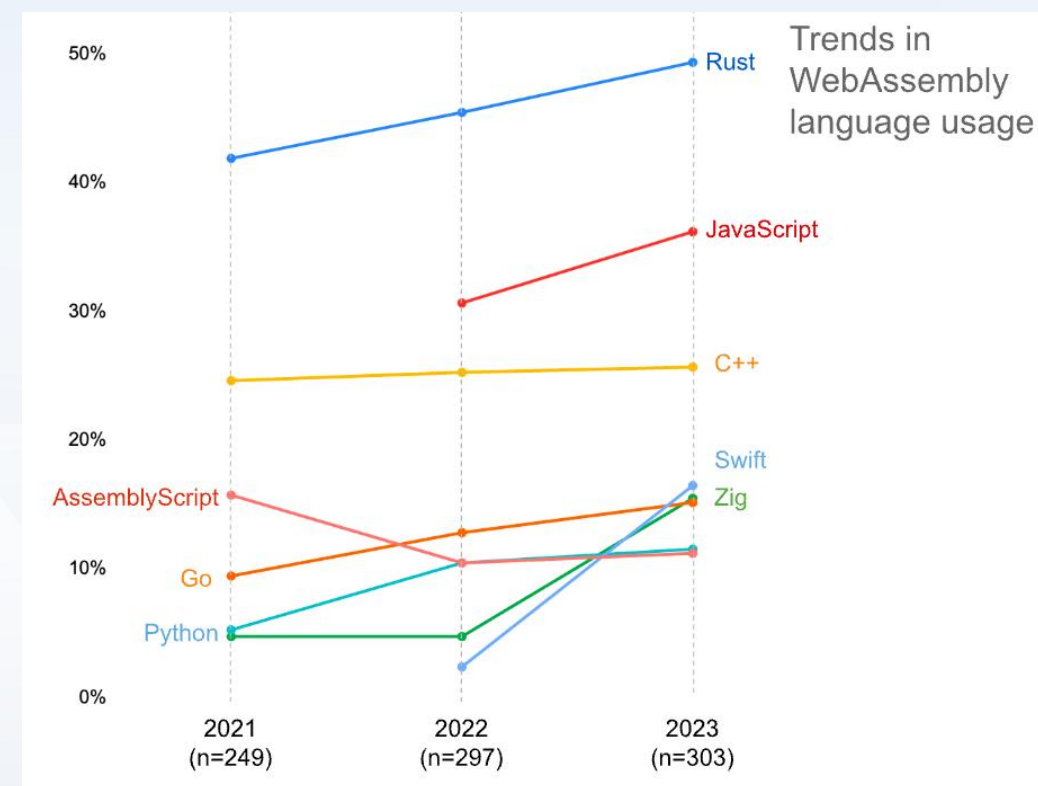
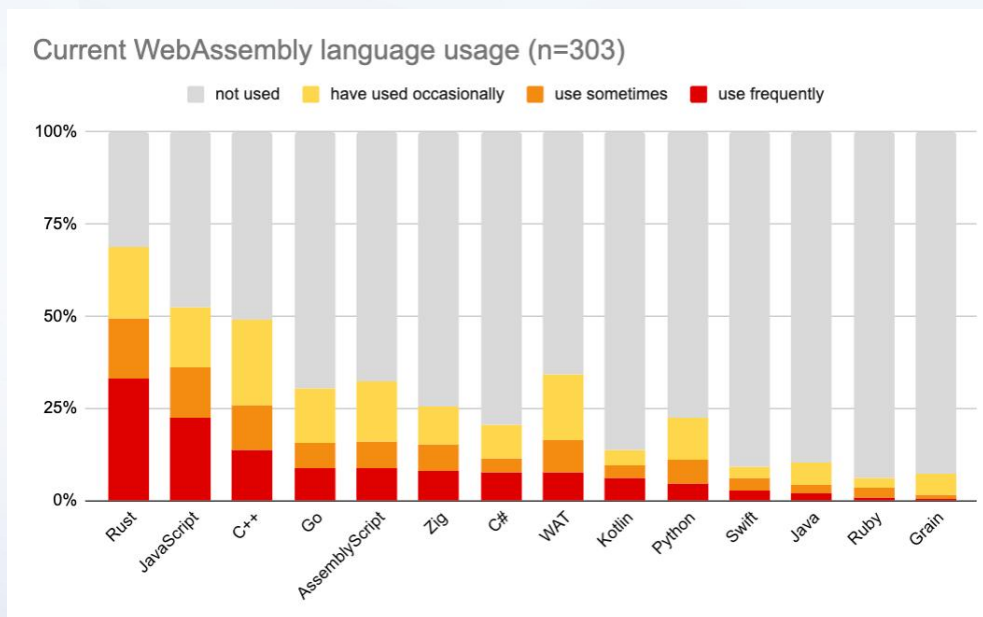
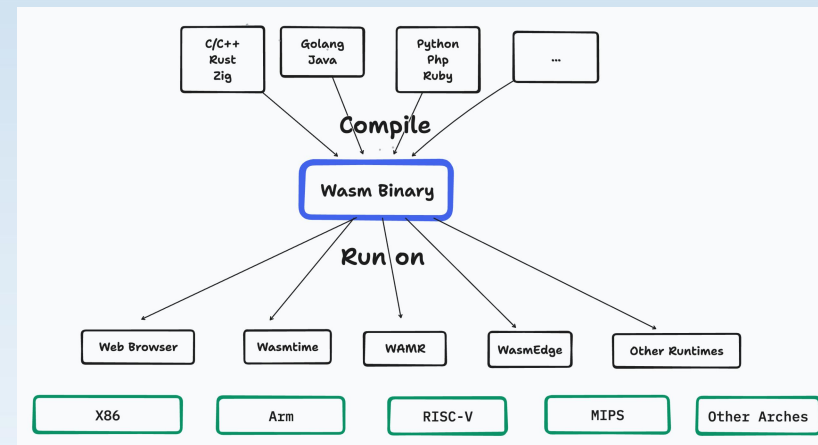
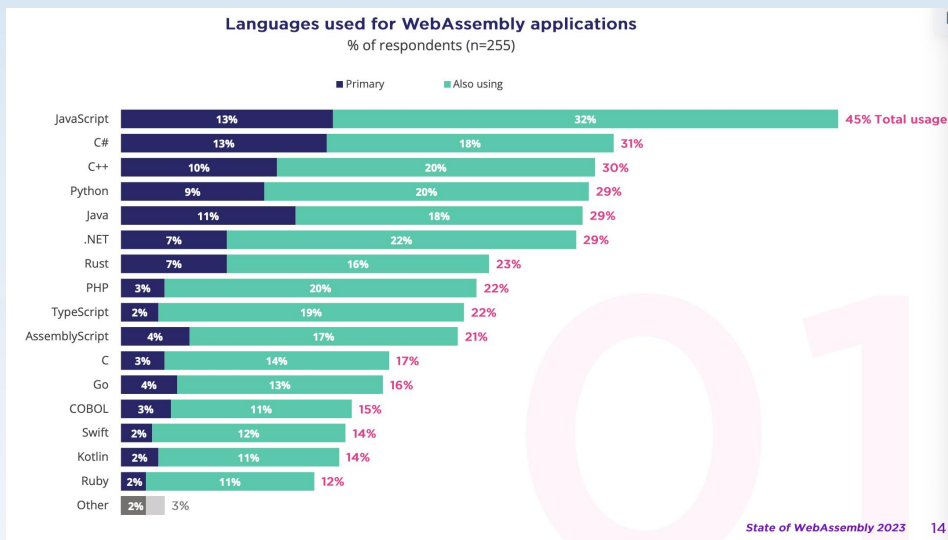
```
(export "memory" (memory 0))
(export "test" (func 1))
(export "main" (func 2))
(export "__data_end" (global 1))
(export "__heap_base" (global 2))
```

```
$ wasm-tools print ./test.wasm | grep "(export"
```

Run with runtime



作为其他语言的跨平台编译目标



为 WASM 而生的语言



Grain

<https://grain-lang.org/>



MoonBit

<https://www.moonbitlang.com/>



凹语言

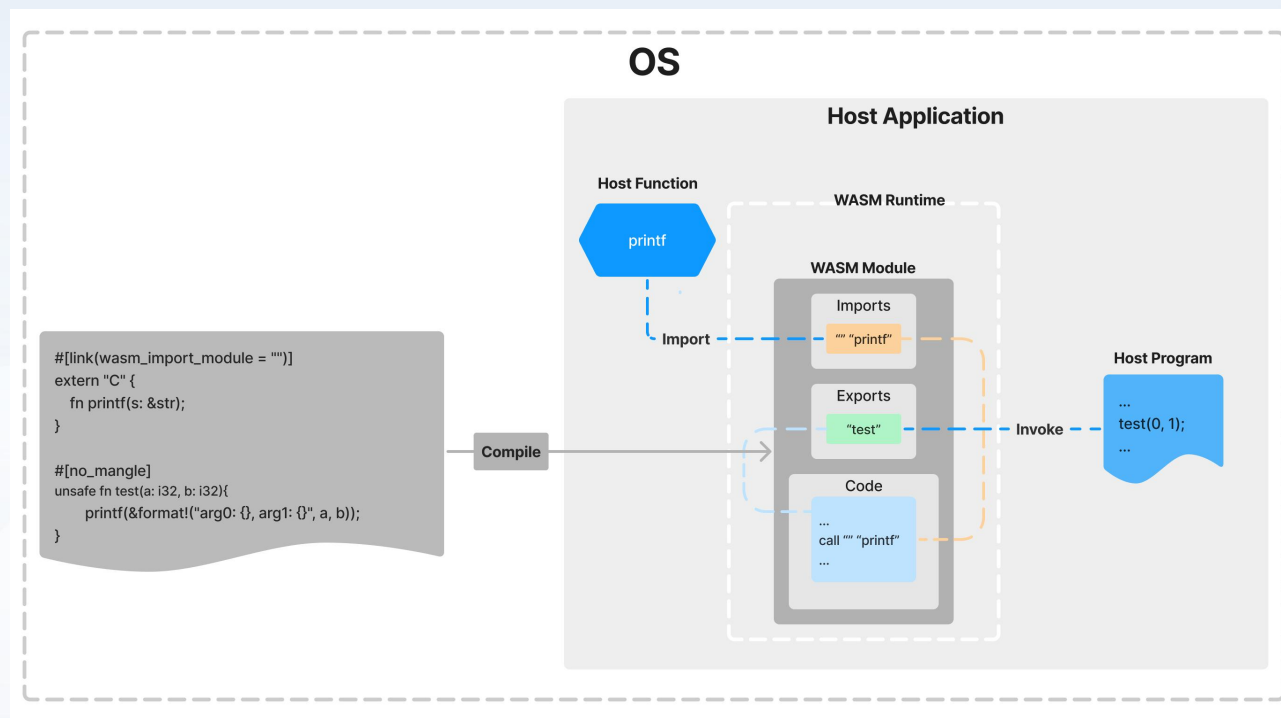
<https://wa-lang.github.io/>

安全沙箱与 Host Functions

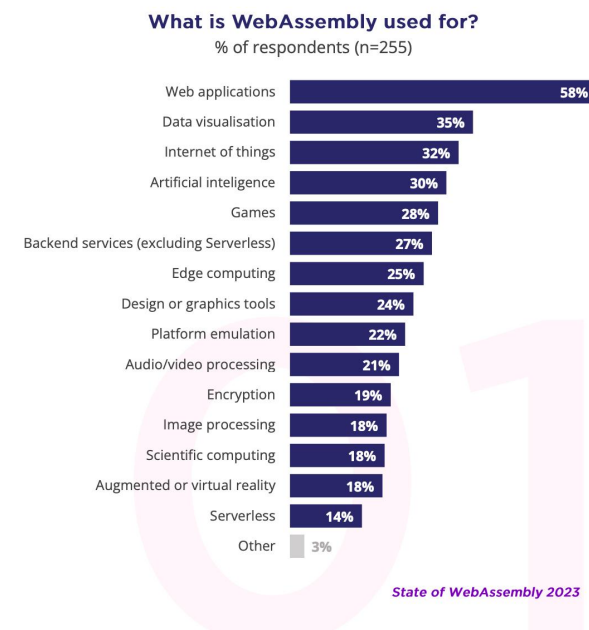
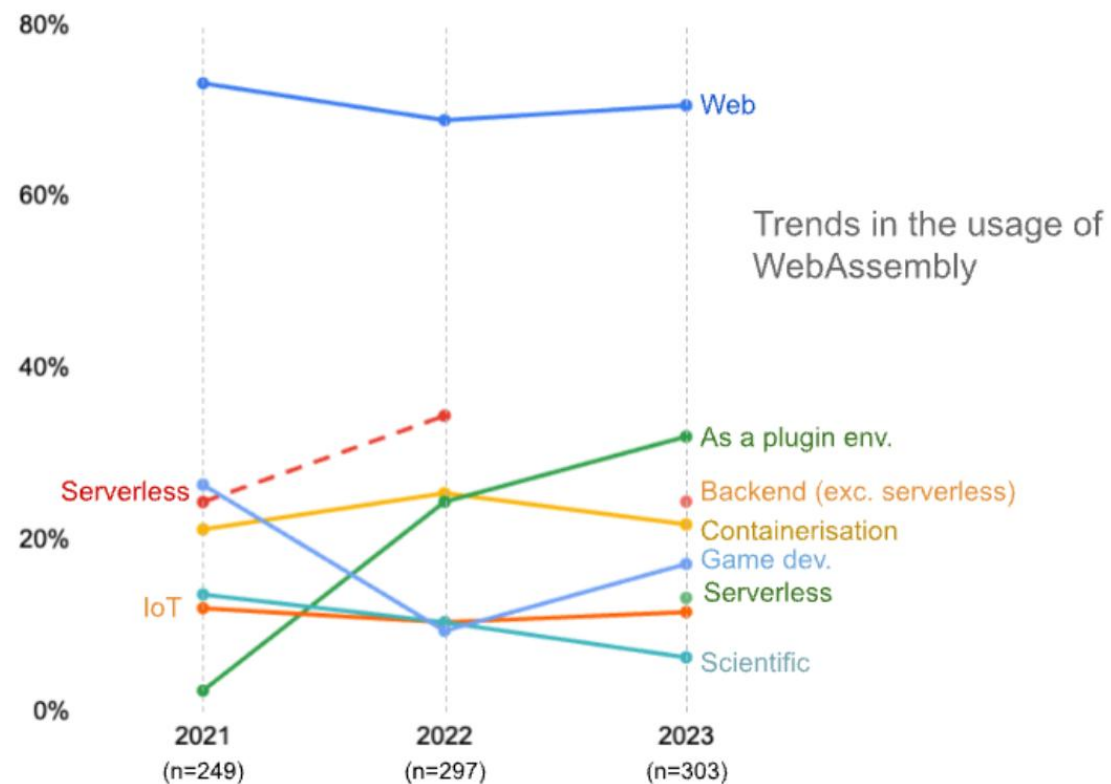
安全沙箱

- 基于栈虚拟机的内存隔离
- 受限的系统功能,不提供对宿主环境的访问。

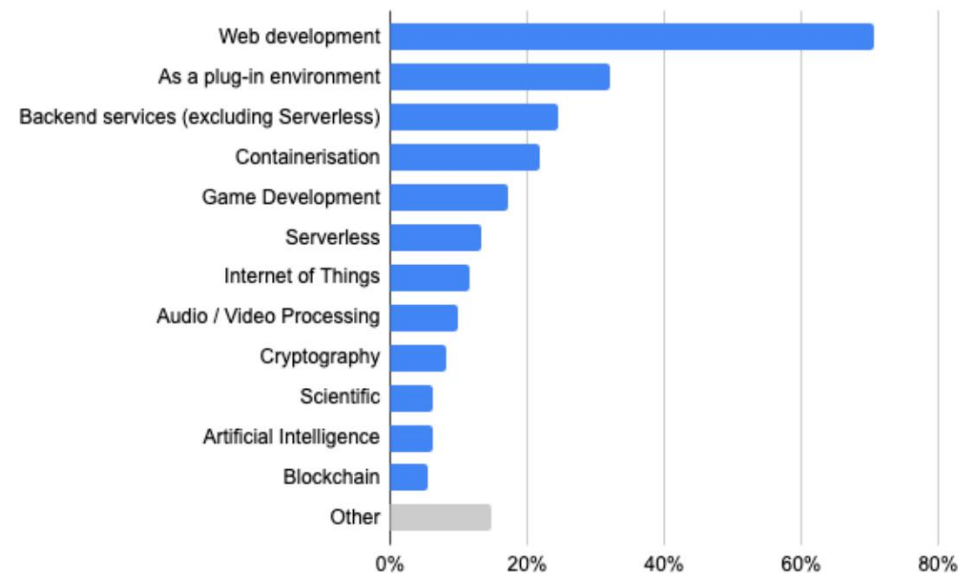
Import Host Functions



Wasm 应用场景



WebAssembly applications (n=303)





Part 03

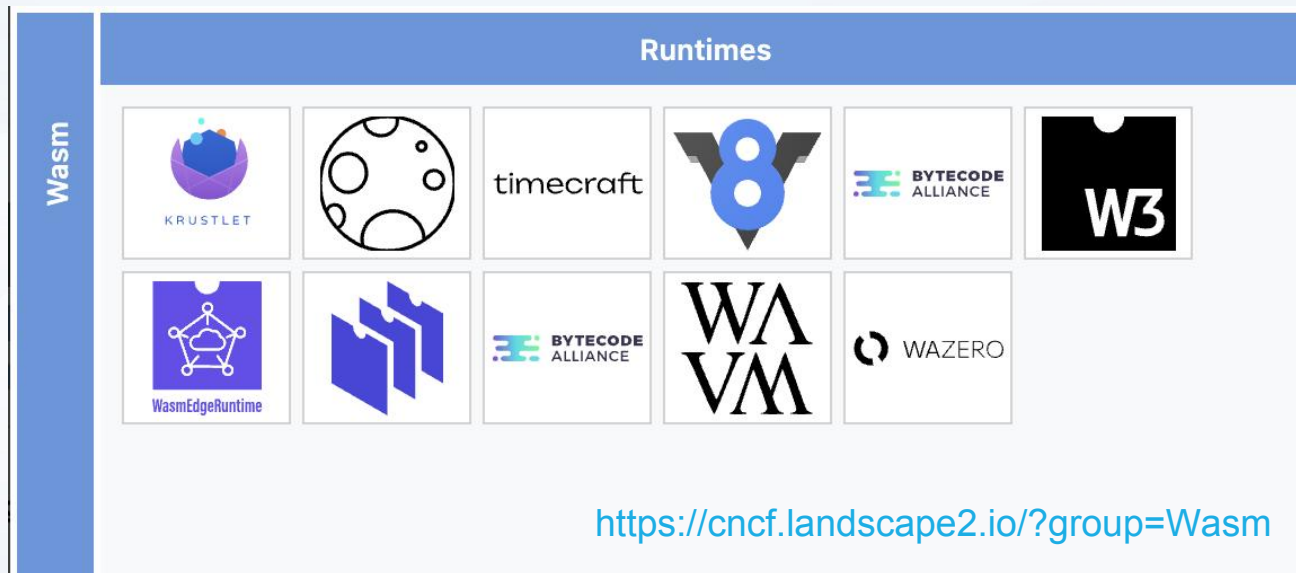
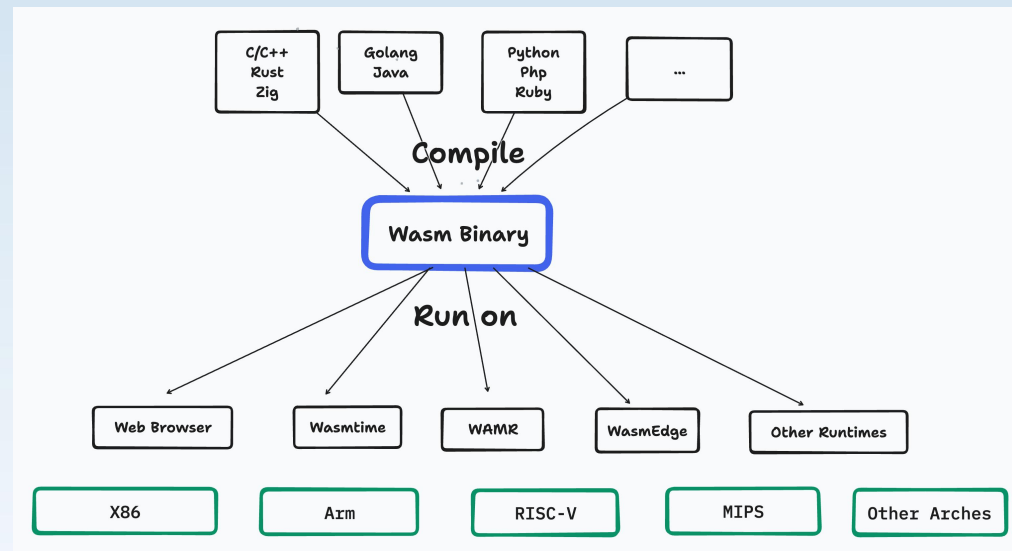
Wasm 探索浏览器外的发展

Wasm Runtimes

- **wasmtime**: Bytecode Alliance 旗下，对于 WASM 标准支持及时
- **wasmedge**: 丰富的周边功能与，支持广泛的操作系统和硬件平台
- **wasmer**: 丰富的周边生态，支持 WASIX 接口，集成库稳定
- **wamr(WebAssembly Micro Runtime)**: Bytecode Alliance 旗下，支持广泛的操作系统和硬件平台，极轻量，可达 50k
- **wazero**: Golang 友好

执行模式

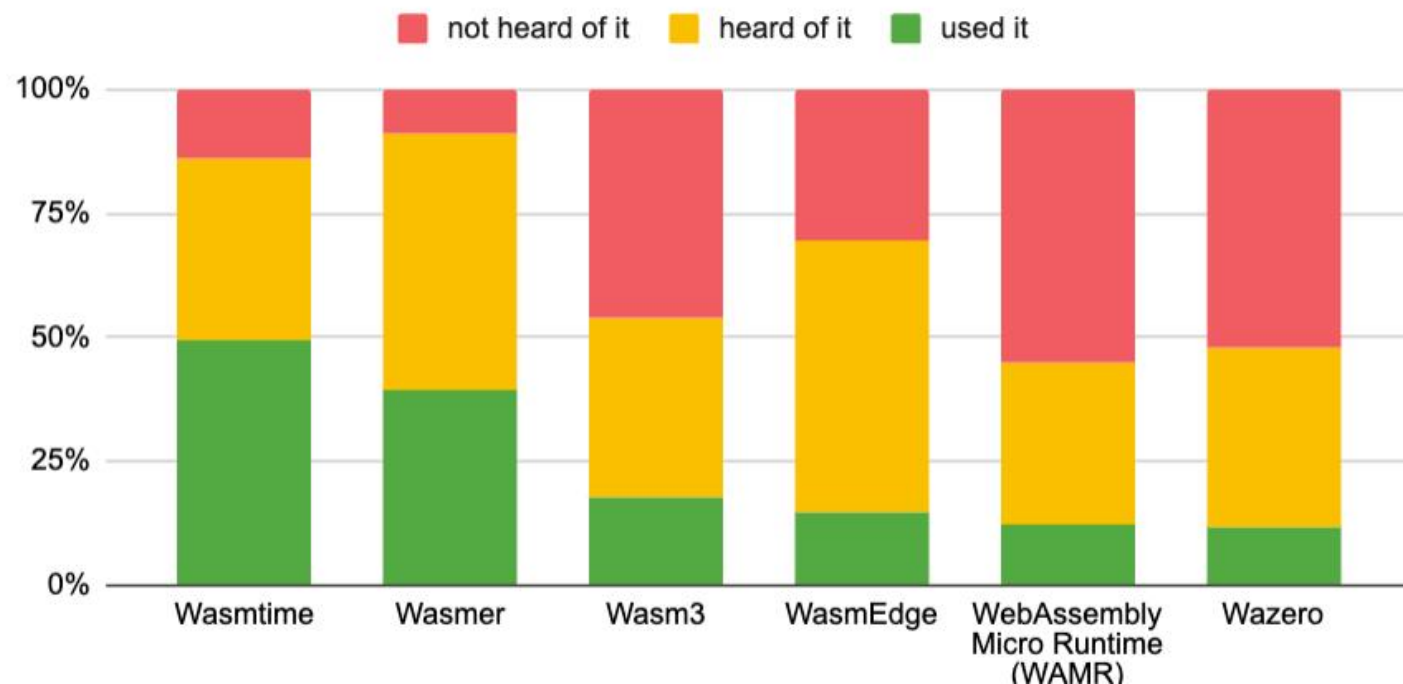
- Interpreted
- JOT
- AOT



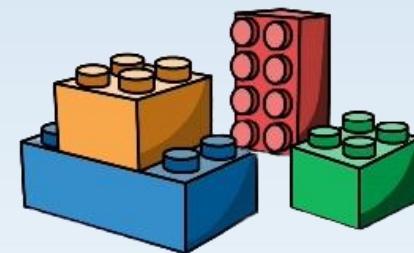
选择合适的运行时

- 性能 (Interpreters, JIT, AOT)
- 宿主应用的语言选择, 和运行时库支持情况 (C/C++, Rust, Go ...)
- 平台架构 (x86, ARM, RISC-V, MIPS, ...)
- 系统要求 (Linux/VxWorks/RT-Thread, ...)
- 社区活跃度与理念

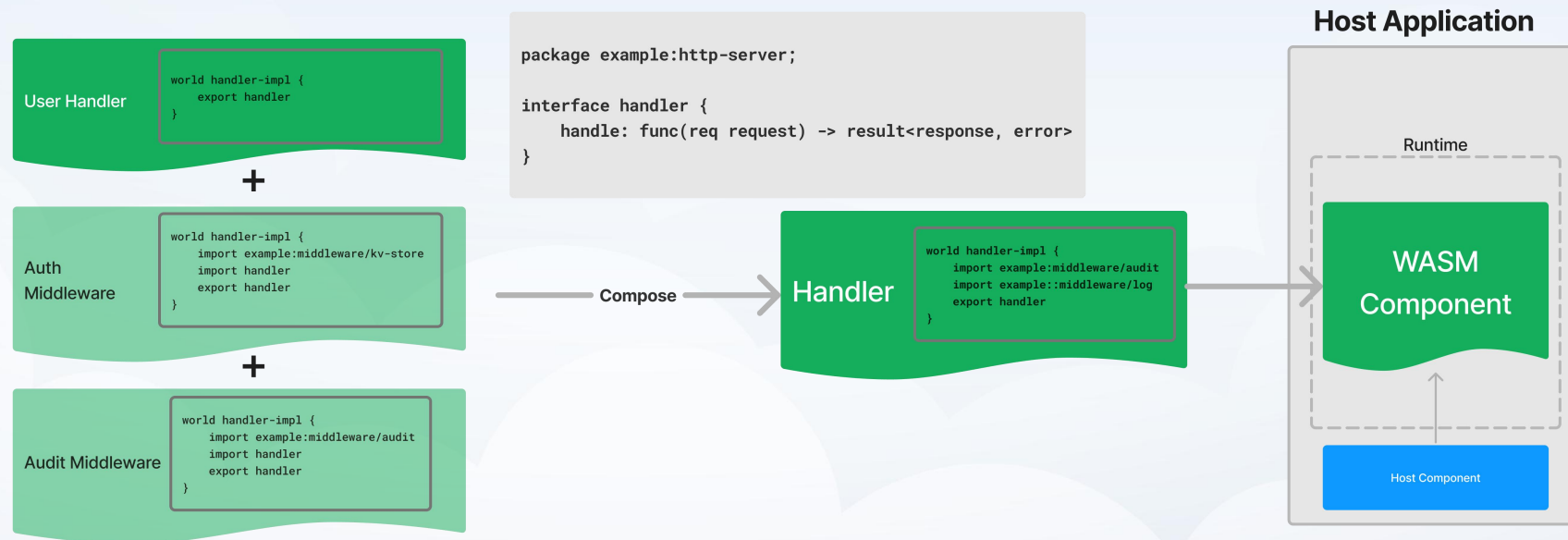
WebAssembly runtime usage



Component Model: World of 'LEGO'



- WIT 接口格式
- 更细粒度的 Wasm 模块



Wasm System Interface

- Webassembly Community Group 制定的一组与其主机环境功能相关的**标准接口规范**
- **与实现无关**，由运行时提供具体的实现

```
(module $wasi_snapshot_preview1
  ;;; Linear memory to be accessed by WASI functions that need it.
  (import "memory" (memory))
  (@interface func (export "args_get")
    (param $argv (@witx pointer (@witx pointer u8)))
    (param $argv_buf (@witx pointer u8))
    (result $error (expected (error $errno)))
  )
  ;;; Return command-line argument data sizes.
  (@interface func (export "args_sizes_get")
    ;;; Returns the number of arguments and the size of the argument string
    ;;; data, or an error.
    (result $error (expected (tuple $size $size) (error $errno)))
  )
)
```

WASM Module

```
interface environment {
  /// Get the POSIX-style environment variables.
  ///
  /// Each environment variable is provided as a pair of string variable names
  /// and string value.
  ///
  /// Morally, these are a value import, but until value imports are available
  /// in the component model, this import function should return the same
  /// values each time it is called.
  get-environment: func() -> list<tuple<string, string>>;

  /// Get the POSIX-style arguments to the program.
  get-arguments: func() -> list<string>;

  /// Return a path that programs should use as their initial current working
  /// directory, interpreting `.` as shorthand for this.
  initial-cwd: func() -> option<string>;
}
```

Component Model

Wasm System Interface

<https://github.com/WebAssembly/WASI/blob/main/Proposals.md>

Phase 3 - Implementation Phase (CG + WG)

Proposal	Champion
I/O	Dan Gohman
Clocks	Dan Gohman
Random	Dan Gohman
Filesystem	Dan Gohman
Sockets	Dave Bakker
CLI	Dan Gohman
HTTP	Piotr Sikora, Jiaxiao Zhou, Dan Chiarlone, David Justice, Luke Wagner

Phase 2 - Proposed Spec Text Available (CG + WG)

Proposal	Champion
Machine Learning (wasi-nn)	Andrew Brown and Mingqiu Sun

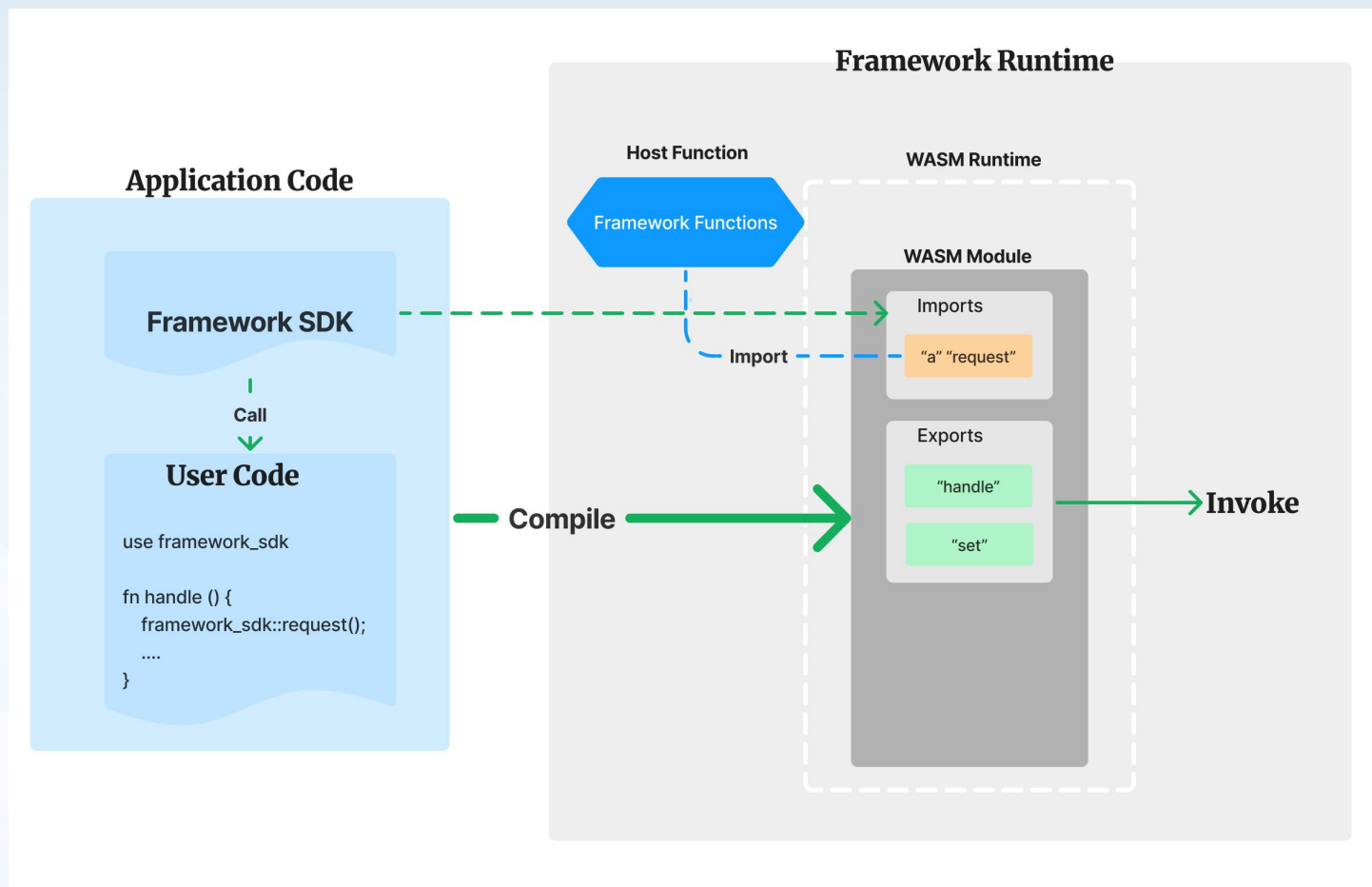
Phase 1 - Feature Proposal (CG)

Proposal	Champion
Blob Store	Jiaxiao Zhou, Dan Chiarlone, David Justice
Crypto	Frank Denis and Daiki Ueno
Digital I/O	Emiel Van Severen
Distributed Lock Service	Jiaxiao Zhou, Dan Chiarlone, David Justice
I2C	Friedrich Vandenbergh
Key-value Store	Jiaxiao Zhou, Dan Chiarlone, David Justice
Logging	Dan Gohman
Message Queue	Jiaxiao Zhou, Dan Chiarlone, David Justice
Parallel	Andrew Brown
Pattern Match	Jianjun Zhu
Pub/sub	Jiaxiao Zhou, Dan Chiarlone, David Justice
Runtime Config	Jiaxiao Zhou, Dan Chiarlone, David Justice
SPI	Emiel Van Severen
SQL	Jiaxiao Zhou, Dan Chiarlone, David Justice
SQL Embed	Kyle Brown
Threads	Alexandru Ene, Marcin Kolny, Andrew Brown
URL	Radu Matei

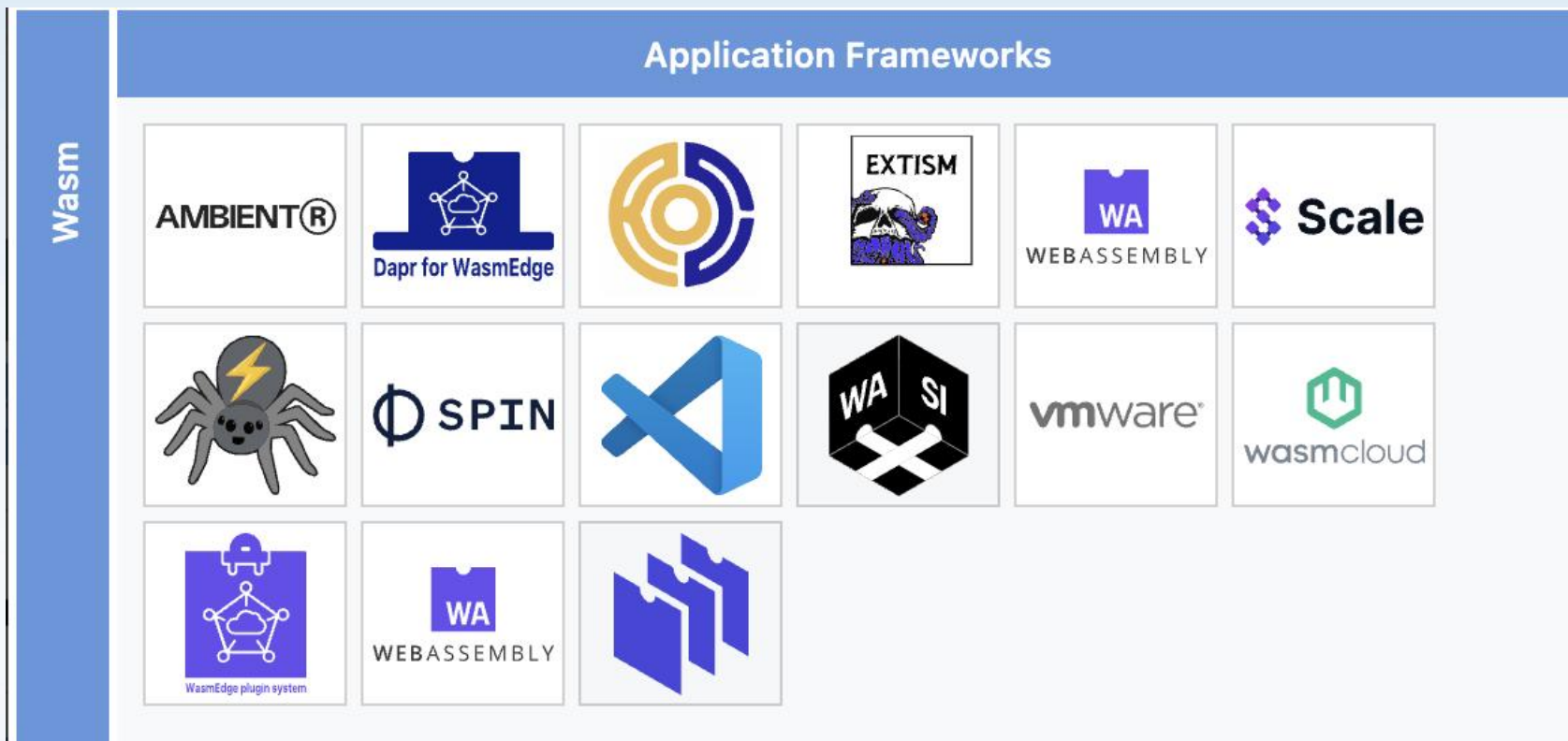
沙箱 与 Wasi 与 应用框架

运行时为 **WASM** 提供了安全的沙箱环境，并且（通常）也会为用户实现 **标准的 WASI 接口**，但是用户的业务应用往往可能会需要更多的能力

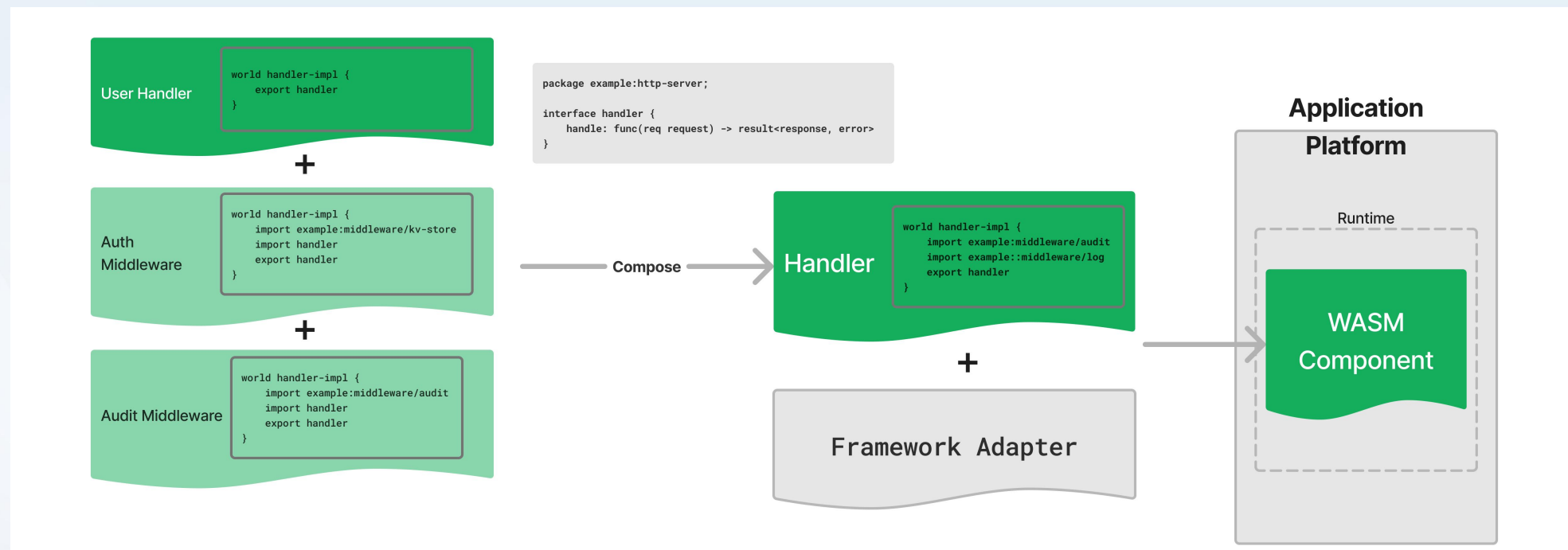
而这些功能便由 **应用框架** 来提供与业务相关的功能，用户根据业务需求来选择合适的框架



Wasm Landscape: 应用框架



应用框架与 Component Model





Part 03

Wasm 生态&组织

Webassembly Work Group: 规范的标准化的

<https://github.com/WebAssembly>

主要是指指定 Wasm 相关规范的标准化的, 除了已经发布的 1.0 的 Wasm 规范, 还有 2.0 草案, 以及相关功能的推进, 其中 对于浏览器外生态建设最具吸引力的功能是 Component Model 与 WASI 标准



Bytecode Alliance (字节码联盟)



BYTECODE ALLIANCE

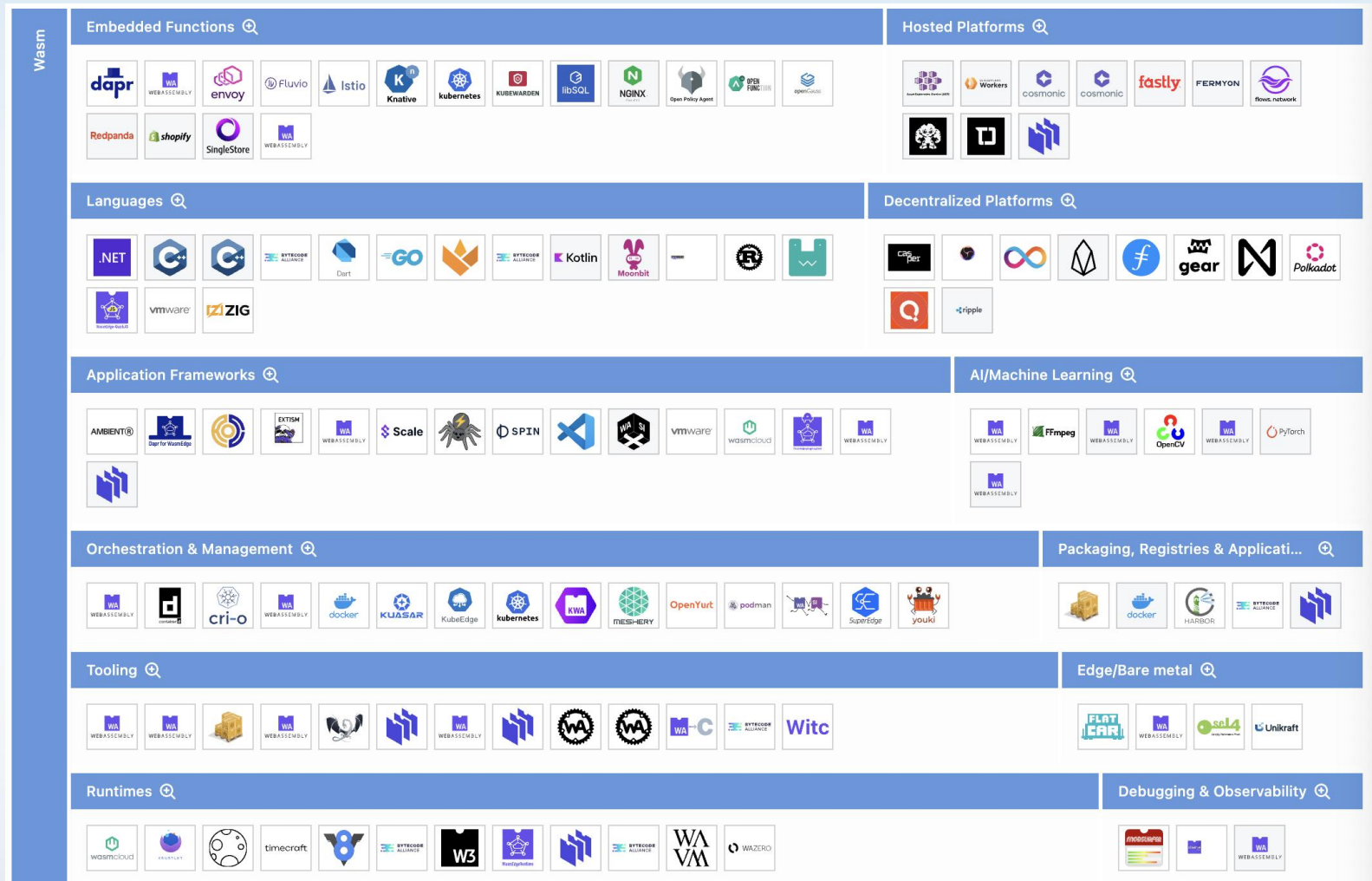
<https://github.com/bytecodealliance>

主要推动 Wasm 在浏览器外发展，主要维护

- 运行时：wasmtime 与 wasm-micro-runtime (WAMR)
- 编译后端：Craneflirt
- 相关工具链

WASM Landscape

<https://cncf.landscape2.io/?group=Wasm>



Wasm 与云原生: CNCF TAG-RUNTIME - Wasm WG

演讲预告

Bridging Wasm and Kubernetes

2:30 PM — 3:00 PM

- Why Wasm + K8s
- Quick reflection on tech like Krustlet
- Wasm working like any other pod workload
- The future of an architecture and OS agnostic OCI package and workload
- “How can you help build this future?”



David Justice
Microsoft

Co-Chair of Wasm WG



Thanks.

